

Protocol Analysis Report

Report type	Network protocol analysis, controlled lab
Version / date	1.0 / 2026-10-06
Capture window (UTC)	2026-10-03 20:38 - 21:00 and 2026-10-05 14:00 - 14:08
Data analyzed	Zeek logs (conn, dns, http, dhcp, ntp, packet_filter) and evidence PCAPs (beacon, dns, port4444)
Not available	evidence-exfil.pcapng (0 bytes when uploaded), attack.pcapng
Related files	findings.md, alert-review.md, alert-review.csv, iocs.csv

1. Executive Summary

This report looks at every protocol that appeared on the lab network and says, for each one, whether it behaved normally or not. The lab contains a Windows victim host and a Kali Linux analysis host. Four behaviors were created on purpose to look like real attack traffic.

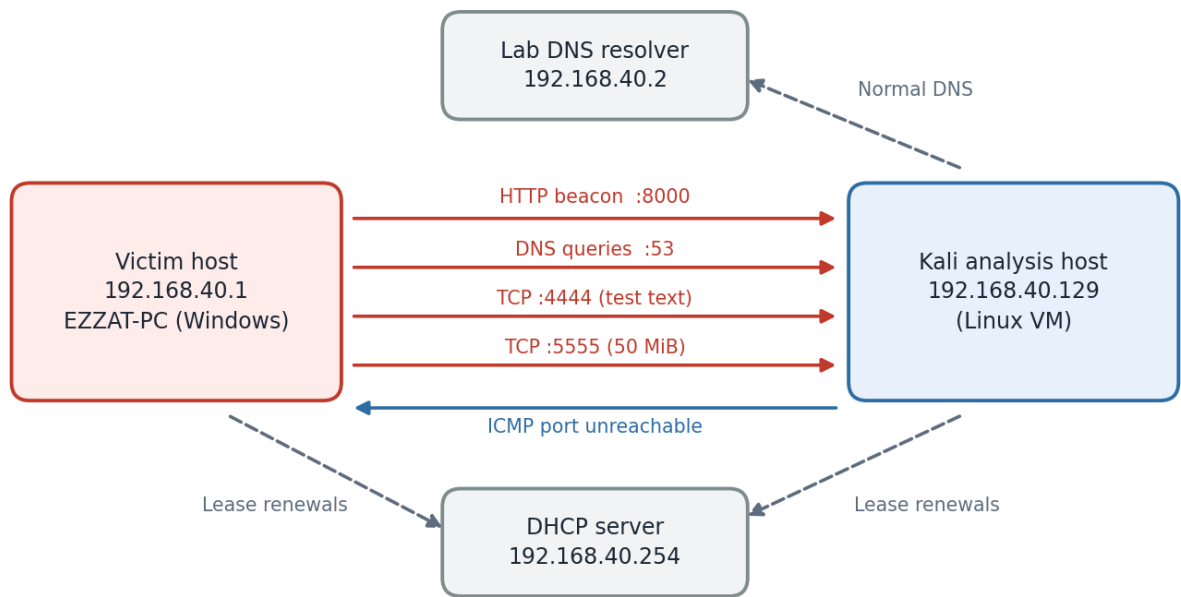
Bottom line: 165 connections were logged. About 60% (99 connections) is routine background traffic such as time sync, normal DNS and browser updates. The other 66 connections (40%) belong to four unusual behaviors, all started by the victim host toward the Kali host.

#	What happened	Protocol	Plain-language meaning	Finding
1	30 identical web requests exactly 30 seconds apart	HTTP, TCP/8000	A program is "checking in" on a timer, like malware beaconing to its controller	F-001
2	33 DNS queries for malicious-c2.xyz using random-looking names	DNS, UDP/53	The pattern resembles hidden data or commands carried in DNS; nothing answered, so no data moved	F-002
3	One 50 MiB transfer in about 1 second	TCP/5555	A large file moved to another host, which is how data theft can look	F-003
4	A session on port 4444 that sent the text LAB_TEST three times	TCP/4444	A port often used by attack tools, but here it carried only harmless test text	F-004

All four match the lab exercise, so none is a real intrusion. The report treats them as if they were real, to show how each one would be recognized.

2. Environment and Data Sources

2.1 Network layout



2.2 Hosts

Host	IP	Hostname	Evidence for its role
Victim	192.168.40.1	EZZAT-PC	TTL 128 and Windows-style TCP options; starts every unusual flow
Kali analysis host	192.168.40.129	none	TTL 64 and Linux-style TCP options; runs the listeners on 8000, 4444 and 5555
DNS resolver	192.168.40.2	none	Answers normal lookups with NOERROR
DHCP server	192.168.40.254	none	Acknowledges lease renewals

2.3 How the data was analyzed

Source	Used for
Zeek logs	Connection counts, protocol mix, timing, DHCP and NTP details
Evidence PCAPs	Packet payloads, TCP/IP header details, exact timing, HTTP content, DNS query contents
Method	Zeek logs were read directly; the PCAPs were parsed with Python (Scapy) and checked against Zeek

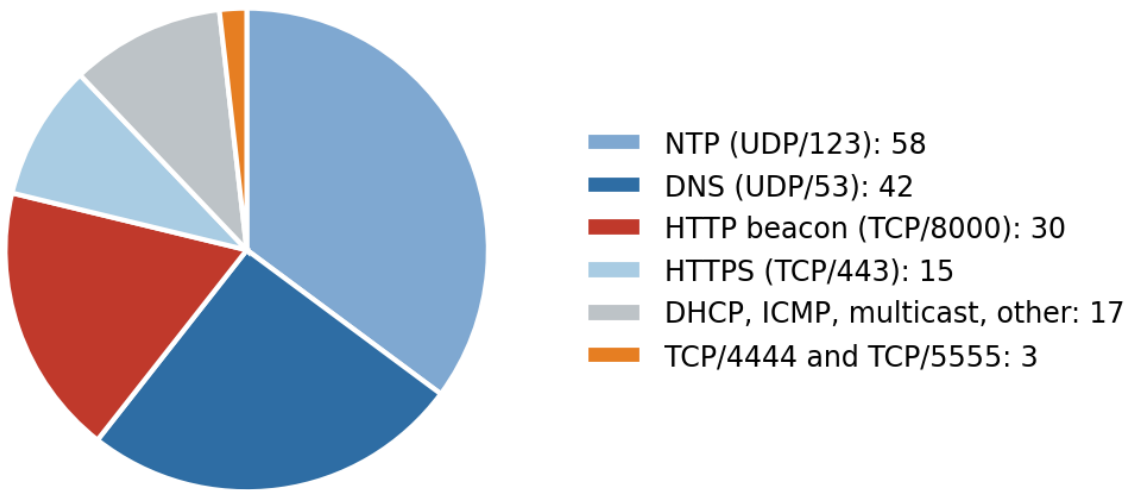
3. Traffic Overview

Zeek logged **165 connections** between 2026-10-03 20:38:11 and 2026-10-05 14:06:03 UTC.

3.1 Protocol mix

Protocol / port	Connections	Share	What it is	Verdict
UDP/123	58	35.2%	NTP time sync	Normal
UDP/53	42	25.5%	DNS	Mixed (see 4.2)
TCP/8000	30	18.2%	HTTP to Kali host	Suspicious
TCP/443	15	9.1%	HTTPS (Mozilla/Fastly)	Normal
UDP/67	5	3.0%	DHCP	Normal
ICMP / ICMPv6	5	3.0%	Echo, unreachable, multicast reports	Mixed (see 4.6)
UDP/5355	4	2.4%	LLMNR name queries	Normal (note risk)
TCP/4444	2	1.2%	Unusual port	Suspicious
IGMP	2	1.2%	Multicast membership	Normal
TCP/5555	1	0.6%	Large transfer	Suspicious
UDP/5353	1	0.6%	mDNS	Normal

Connections by protocol (165 total)



The four unusual behaviors account for 66 of the 165 connections (30 beacon requests, 33 DNS flows to the Kali host, and 3 on ports 4444 and 5555) and carry nearly all the data: the single port 5555 flow holds over 99.9% of all bytes sent by clients.

3.2 Connection states

Zeek labels how each connection started and ended. Many of these logs show partial states because the sensor often saw only one direction of the traffic.

State	Count	Meaning
SHR	72	Only the responder's side was seen (mostly NTP replies)
S0	39	Connection attempt with no reply seen
SH	31	Only the originator's side was seen (the 30 beacon requests and one more)
OTH	19	Traffic seen mid-connection, no handshake observed
SF	2	Normal start and finish
RSTO	2	The originator reset the connection

4. Protocol-by-Protocol Analysis

4.1 HTTP — TCP/8000 (suspicious, F-001)

In plain terms: HTTP is how web pages are requested. Here, a script asks the Kali host for `/beacon` every 30 seconds, like a device repeatedly "phoning home."

Attribute	Value
Client → server	192.168.40.1 → 192.168.40.129:8000
Requests	30, from 2026-10-03 20:45:49 to 21:00:20 UTC
Request line	GET /beacon HTTP/1.1, identical every time (162 bytes)
User-Agent	python-requests/2.34.2 (a script, not a browser)
Other headers	Accept: /*, Accept-Encoding: gzip, deflate, zstd, Connection: keep-alive; no Referer or cookies
Connections	A new TCP connection (new source port) for every request: 30 requests, 30 ports
Server reply	The same 335-byte HTML "404 File not found" page each time, in Python <code>http.server</code> style
Server behavior	The reply carries the FIN flag, so the server closes each connection after answering

Timing between requests (29 gaps)

Minimum	Maximum	Mean	Median	Standard deviation
29.77 s	30.42 s	30.03 s	30.03 s	0.11 s

Assessment: a fixed cadence with under 0.4% variation, a script user agent and an unchanging URI are classic beacon traits. Because the server only returns 404, no commands or data were exchanged. Zeek recorded no HTTP status codes or response sizes for these requests, while the PCAP shows the full replies.

4.2 DNS — UDP/53 (mixed, F-002)

In plain terms: DNS turns names into addresses. Normally a host asks its resolver. Here the victim asked the Kali host directly for strange, random names, which is a pattern attackers use to hide data inside DNS.

Zeek's `dns.log` holds 52 records, which break down as follows.

Group	Records	Source → destination	Assessment
malicious-c2.xyz activity	33	192.168.40.1 → 192.168.40.129:53	Suspicious
Normal lookups answered by the resolver	10	192.168.40.129 → 192.168.40.2:53	Normal
LLMNR name queries	8	192.168.40.1 → multicast :5355	Normal (see 4.8)

Group	Records	Source → destination	Assessment
mDNS query	1	192.168.40.1 → 224.0.0.251:5353	Normal

Suspicious DNS in detail

Attribute	Value
Window	2026-10-05 14:03:41 – 14:06:03 UTC
Queries	33 = 26 address lookups (A/AAAA) for 7 names + 7 reverse lookups (PTR) of 129.40.168.192.in-addr.arpa
Pattern	Each run starts with a PTR, then A, AAAA, A, AAAA about 2 s apart; 7 runs about 10.5 s apart
Answers	None. All 33 queries drew an ICMP "port unreachable" reply, so nothing listens on UDP/53 of the Kali host
Recursion	RD (recursion desired) set on every query

The pattern of "PTR first, then retries every 2 seconds" is how the `nslookup` tool behaves when run against a server that does not answer.

Names queried

Name (under .malicious-c2.xyz)	Label length	Entropy (bits/char)
random-subdomain	16	3.38
lxpht4i9n7vqug5w3mfr	20	4.32
t7d2m1i6ycja8hbeg54z	20	4.32
f83s5xveji7tn1q2ugbr	20	4.32
l7fthbajv4q9p32mcoys	20	4.32
m9yjsbv2rk0a734qtngz	20	4.32
capf0hr7x65bqn981duw	20	4.32

Assessment: long random labels can carry encoded data, which is how DNS tunneling works. Here every 20-character label uses 20 different characters, which a real encoding would rarely produce, so the names look script-generated. No answers came back, so no data exchange is shown.

Other DNS items

Observation	Detail
Hex-label lookups	On 2026-10-03 at 20:43:30–35 the Kali host asked the resolver for 8f72a91c32, 92ad81c72f and a7812bc981 under example.com (A and AAAA, 1 s apart); all got empty NOERROR replies
Zeek gap	Zeek logged these replies without a query name; the PCAP shows the names
Normal baseline	ads.mozilla.org and push.services.mozilla.com resolved normally

4.3 TCP port 4444 (suspicious, F-004)

In plain terms: Port 4444 is the default for several hacking tools, so it gets attention. In this session, though, the victim sent only the words `LAB_TEST` three times.

Attribute	Value
Connection	192.168.40.1:48924 → 192.168.40.129:4444
Start	2026-10-03 20:38:41 UTC, full three-way handshake
Length	204.9 s, with no FIN or RST (the session stayed open)

Attribute	Value
Data sent	3 messages of 9 bytes each, LAB_TEST\n (27 bytes)
Message times	+43.3 s, +76.8 s, +204.9 s (irregular, not a timer)
Server data	None; the server only acknowledged

Offset	Direction	TCP flags	Content
0.0 s	Victim → Kali	SYN	—
0.0 s	Kali → Victim	SYN, ACK	—
0.0 s	Victim → Kali	ACK	—
43.3 s	Victim → Kali	PSH, ACK	LAB_TEST
76.8 s	Victim → Kali	PSH, ACK	LAB_TEST
204.9 s	Victim → Kali	PSH, ACK	LAB_TEST

Assessment: the port is notable but the content is plain labelled text, with no shell prompt, commands or binary data. Zeek split this one session into two log records (an `SO` and an `OTH` of 161.6 s); the PCAP shows a single 204.9 s connection.

4.4 TCP port 5555 (suspicious, F-003)

In plain terms: One short connection pushed exactly 50 MiB from the victim to the Kali host, the size of a typical test file.

Attribute	Value (from Zeek conn.log)
Flow	192.168.40.1:65137 → 192.168.40.129:5555
Time	2026-10-03 about 20:52:59 UTC
Duration	0.99 s
Client → server bytes	52,428,800 (exactly 50 MiB)
Server → client bytes	0
Packets recorded	56 packets, 75,900 IP bytes

Assessment: `evidence-exfil.pcapng` was not readable, so this rests on Zeek only. 56 packets cannot carry 50 MiB, and Zeek computes bytes from TCP sequence numbers, so the capture probably holds only part of the flow. The packet-level check is still pending.

4.5 TCP/IP details shared by the PCAP sessions

These low-level details back up the host roles and show the traffic came from two real, different systems.

Detail	Victim (192.168.40.1)	Kali host (192.168.40.129)
IP TTL	128 (Windows default)	64 (Linux default)
SYN window	65,535	64,240 (SYN-ACK)
TCP options	MSS 1460, window scale 8, SACK	MSS 1460, window scale 8, SACK
IP ID	Increments per packet (34922, 34923, ...)	0 on SYN-ACK (Linux behavior with Don't Fragment)
Don't Fragment	Set	Set

4.6 ICMP, ICMPv6 and IGMP

In plain terms: ICMP carries network "status messages" such as ping and "nobody is listening here."

Event	Time (UTC)	Detail	Assessment
Echo request and reply (ping)	2026-10-05 14:02:57	192.168.40.1 ↔ 192.168.40.129, 128 bytes each way	Reachability check just before the DNS activity
Port unreachable (type 3, code 3)	2026-10-05 14:03:41 – 14:06:03	33 messages from 192.168.40.129 to 192.168.40.1, one per DNS query	Explains why the C2 domain never got an answer
MLD reports (ICMPv6 type 143)	2026-10-03 20:42:17, 20:57:26	fe80::5742:1fbe:ce5a:ea5a → ff02::16	Normal Windows multicast behavior
IGMP membership	same two times	192.168.40.1 multicast join reports	Normal

4.7 NTP — UDP/123 (normal)

In plain terms: NTP keeps clocks accurate. The Kali host polls a time server about every 30 seconds.

Server	Exchanges	Window (UTC)
196.44.136.165	13	2026-10-03 20:38:14 – 20:44:14
196.10.55.57	31	2026-10-03 20:45:24 – 21:00:14
196.10.54.57	13	2026-10-05 14:00:00 – 14:06:00

All 57 exchanges used NTP version 4, mode 4 (server reply) and stratum 2, with a median gap of about 30 seconds. The server changed between sessions, which is consistent with the lab's time-server pool changing. The reference-time fields contain unusual values but nothing else is abnormal, so this is treated as noise.

4.8 LLMNR and mDNS (normal, with a known risk)

In plain terms: When Windows cannot find a name through DNS, it shouts the question to the local network.

Protocol	Port	Observation
LLMNR	UDP/5355	ezzatzpc queried over IPv4 (224.0.0.252) and IPv6 (ff02::1:3) at 20:42:17 and 20:57:26 UTC; 8 records in dns.log
mDNS	UDP/5353	_sleep-proxy._udp.local PTR query at 20:49:39 UTC

This is normal Windows behavior. LLMNR can be abused by attackers on real networks to capture credentials, so many environments disable it.

4.9 DHCP — UDP/67 (normal)

Time (UTC)	Client	Message	Lease
2026-10-03 20:38:14	192.168.40.129	ACK	1800 s
2026-10-03 20:42:17	192.168.40.1 (EZZAT-PC)	REQUEST, ACK	1800 s
2026-10-03 20:53:14	192.168.40.129	ACK	1800 s
2026-10-03 20:57:26	192.168.40.1 (EZZAT-PC)	REQUEST, ACK	1800 s
2026-10-05 14:06:00	192.168.40.129	ACK	1800 s

The server is 192.168.40.254. Renewals arrive about 900 s apart, half the 30-minute lease, which is standard client behavior. This log is also where the hostname EZZAT-PC comes from.

4.10 HTTPS — TCP/443 (normal)

15 flows from the Kali host went to 199.232.81.91 (6 flows) and 34.107.243.93 (9 flows), the same addresses returned for the Mozilla DNS lookups. This looks like Firefox background traffic. Several flows show reset or mid-stream states because the capture did not see complete sessions. The content is encrypted, so only metadata could be reviewed.

5. Cross-Protocol Timeline

Time (UTC)	Protocol	Event
2026-10-03 20:38:14	DHCP, NTP	Kali host renews its lease and starts polling time
2026-10-03 20:38:41	TCP/4444	Handshake to Kali host
2026-10-03 20:39:24	TCP/4444	First LAB_TEST message
2026-10-03 20:43:30	DNS	Hex-label lookups under example.com
2026-10-03 20:45:49	HTTP	First /beacon request
2026-10-03 20:52:59	TCP/5555	50 MiB transfer
2026-10-03 21:00:20	HTTP	30th and last beacon
2026-10-05 14:02:57	ICMP	Ping from victim to Kali host
2026-10-05 14:03:41	DNS	First malicious-c2.xyz run
2026-10-05 14:06:03	DNS	Last captured query
2026-10-05 14:08:10	Zeek	Packet filter started

6. Protocol Risk Matrix

Protocol	Port	Verdict	Severity	Finding	Key evidence
HTTP	TCP/8000	Suspicious, lab-generated	Medium	F-001	30 requests, 30.03 s mean gap, script user agent
DNS	UDP/53	Suspicious, lab-generated	Medium	F-002	Random 20-character labels, queries sent to a non-resolver
TCP	5555	Suspicious, lab-generated	Medium	F-003	50 MiB in 0.99 s (Zeek only)
TCP	4444	Suspicious, lab-generated	Medium	F-004	Default tool port, LAB_TEST payload
ICMP	n/a	Supporting evidence	Info	F-002	33 port-unreachable replies
NTP	UDP/123	Normal	Info	none	Regular 30 s polling
DHCP	UDP/67	Normal	Info	none	Lease renewals every 900 s
LLMNR / mDNS	UDP/5355, 5353	Normal	Info	none	Windows name resolution
HTTPS	TCP/443	Normal	Info	none	Mozilla/Fastly addresses

7. Recommendations

If this traffic appeared on a production network, these steps would apply.

Priority	Protocol	Action
High	DNS	Block or sinkhole <code>malicious-c2.xyz</code> ; allow DNS only to approved resolvers and alert on queries to other internal hosts
High	HTTP	Alert on fixed-interval requests to the same URI, especially from script user agents such as <code>python-requests</code>
High	TCP/5555	Review large outbound transfers to uncommon ports; confirm what was sent
Medium	DNS	Add detection for long, high-entropy subdomain labels and repeated failed lookups
Medium	TCP/4444	Alert on connections to port 4444; identify the process and service on both ends
Medium	LLMNR	Disable LLMNR and NetBIOS name resolution where not needed
Low	Zeek	Check capture completeness (the 50 MiB flow shows only 56 packets) and confirm full HTTP logging

8. Limitations

- `evidence-exfil.pcapng` was empty, so F-003 relies on Zeek metadata only.
- `attack.pcapng` and the lab screenshots were not part of this analysis.
- Zeek often saw one direction of a flow, so some counts, states and byte totals are partial.
- Host roles come from the lab description and are supported by TTL and TCP option evidence; no host forensics were done.
- HTTPS content is encrypted, so only addresses and counts were reviewed.

Appendix A — Wireshark Filters

Goal	Filter
Beacon requests	<code>http.request.uri contains "beacon"</code>
All HTTP to Kali host	<code>tcp.port == 8000</code>
Suspicious DNS	<code>dns.qry.name contains "malicious-c2"</code>
Reverse lookups sent to Kali	<code>dns.qry.type == 12 && ip.dst == 192.168.40.129</code>
ICMP port unreachable	<code>icmp.type == 3 && icmp.code == 3</code>
Port 4444 session	<code>tcp.port == 4444</code>
Large transfer	<code>tcp.port == 5555</code>
Test payload	<code>frame contains "LAB_TEST"</code>

Appendix B — Glossary

Term	Meaning
Beaconing	A program contacting a controller on a regular timer
C2 (command and control)	The channel an attacker uses to direct a compromised host
DNS tunneling	Hiding data inside DNS queries and replies
Entropy	A measure of randomness; higher means more random-looking text
Exfiltration	Moving data out of a network without permission
TTL	A packet counter that differs by operating system (128 Windows, 64 Linux)
Zeek	A network monitor that writes logs about connections and protocols